计算机系统原理实验报告

第二次实验
NEMU 模拟器及 Cache 行为仿真

成员：	江李阳	202400460042
	王健一	202400460037
	邱泽辉	202400460046
	史淇	202333180074
专业：	密码科学与技术	
日期：	April 25, 2026	

Contents

1 实验目的 **2**

2 实验环境 **2**

3 实验原理 **2**
3.1 局部性原理 (Principle of Locality) 2
3.2 Cache 结构与地址拆分 2
3.3 Cache 块映射方式 (Address Mapping) 3
3.4 替换算法 (Replacement Algorithms) 3
3.5 Cache 写策略 (Writing Policy) 4

4 实验过程 **4**
4.1 Cache 大小对命中率的影响 5
4.1.1 实验结果 5
4.1.2 实验现象描述与分析 6
4.1.3 实验总结 7

1 实验目的

学习 NEMU 模拟器的基本功能, 理解 cache 的工作原理, 以及相应参数与算法对程序性能的影响。

2 实验环境

- 硬件环境:
  - 处理器:13th Gen Intel(R) Core(TM) i5-13420H(12CPUs), 2.1GHz
  - 内存:16384MB RAM
- 软件环境:
  - 操作系统:Windows11
  - VMware Workstation Pro 17:Ubuntu 22.04 LTS 系统,7.8GB 内存,4 处理器,300GB 硬盘
  - Arch Linux x86_64(内核:Linux 6.6.87.2-microsoft-standard-WSL2)
    - L1d cache:288KiB(6 instances)
    - L1i cache:192 KiB(6 instances)
    - L2 cache:7.5 MiB(6 instances)
    - L3 cache:11MiB(1 instances)

3 实验原理

3.1 局部性原理 (Principle of Locality)

Cache 存在的理论基础就是局部性原理:

- 时间局部性 (Temporal Locality)：如果一个存储单元被访问，则近期内它很可能再次被访问（例如：循环变量、重复调用的函数）。
- 空间局部性 (Spatial Locality)：如果一个存储单元被访问，则其邻近的存储单元近期内也很可能被访问（例如：数组遍历、顺序执行的指令）。

Cache 通过存放这些“活跃”数据，减少了 CPU 访问高延迟 DRAM 的次数, 大大提高了系统运行效率。

3.2 Cache 结构与地址拆分

假设 Cache 的块的大小为 $B = 2^b$Byte, 共有 $S = 2^s$ 个组, 每组有 W 路, 则对于一个物理地址 Addr, 其逻辑地址被划分为三部分:

$$Addr = [\text{Tag}|\text{Index}|\text{Offset}]$$

- Offset(块内偏移):b 位, 决定数据在 Cache Line 中的具体字节位置。

- Index(组索引):s 位, 用于定位该地址属于哪一个 Set。计算公式为:

$$Index = (Addr >> b)\&(S-1)$$

- Tag(标志位): 剩余的高位, 用于区分映射到同一个组的不同内存块。

3.3 Cache 块映射方式 (Address Mapping)

Cache 地址映射方式决定了主存块如何被放置到 Cache 中, 是 Cache 实现的核心:

- 直接映射 (Direct Mapping)
  - 原理: 每个主存块映射到 Cache 中的唯一固定一行 (每组只有一行)。
  - 计算逻辑:$Cache$行号 $=$ (主存块号) (mod Cache 总行数)
  - 地址划分:
    - Offset: 块内偏移, 由块大小决定。
    - Index: 索引位: 直接对应 Cache 行号。
    - Tag: 高位剩余部分, 用于确定该行存放的是否是目标块。
  - 特点: 硬件电路最简单, 但冲突试效率高。如果两个经常访问的块映射到同一行, 会发生剧烈抖动。
- 全相联映射 (Fully Associative Mapping)
  - 原理: 主存中的任意一块可以放置在 Cache 中的任意一行 (因为只有一个组)。
  - 计算逻辑: 没有 Index 字段，只需匹配 Tag。
  - 地址划分: 只分为 Tag 和 Offset。
  - 特点: 冲突概率最低, 空间利用率最高。但因为全相联高速缓存电路必须并行地搜索许多相匹配的标记, 构造一个又大又快的相联高速缓存很困难, 而且很昂贵, 硬件成本极高, 功耗大, 通常只用作极小容量的存储 (如 TLB)。
- 组相联映射 (Set-Associative Mapping)
  - 原理: 这是前两者的折中。Cache 分为若干个组 (Set)，每组包含 k 个路 (Way)。主存块映射到固定的组，但可以放在该组内的任意一路。
  - 计算逻辑:$SetIndex =$ (主存块号) (mod $Cache$组数)
  - 特点: 兼顾了直接映射的低成本搜索和全相联的低冲突概率。实现时先通过组索引计算组号, 组中任何一行都可以包含任何映射到这一组的内存块, 所以必须搜索每组中每一行, 寻找一个有效的行, 其标记与地址中的标记相匹配。

3.4 替换算法 (Replacement Algorithms)

当 Cache 发生缺失 (Miss) 时, 且计算得到的 Cache 对应位置 (组) 已经满时, 必须决定换出哪一行。

- FIFO(First-In-First-Out, 先进先出)

  - 原理: 总是替换掉最先进入 Cache 的那一块。
  - 实现逻辑: 为每一行维护一个时间戳或使用一个循环队列, 最早调入的块被最先替换掉。
  - 评价: 实现简单, 但忽略了局部性原理, 因为一个很早被调入的块也可能是会被频繁访问的块, 这样一个频繁被访问的块也会被替换掉, 因此该算法性能一般。

- LRU(Least Recently Used, 最近最少使用)
  - 原理: 替换掉距离现在最长时间未被访问的那一块。
  - 实现逻辑: 每次命中某行时, 更新其计数器或移动到链表首部; 发生替换时选择计数器数值最小、反映" 最久远" 或链表尾部的行。
  - 评价:LRU 具体实现会较为复杂, 但能很好地利用局部性原理, 算法性能较优。

3.5 Cache 写策略 (Writing Policy)

Cache 的写操作分为写命中 (write hit) 和写不命中两种情况:

- 写命中:
  - 直写 (write-through): 立即将字 w 的告诉缓存块写回紧接的低一层, 实现简单, 但每次写都会引起总线流量。
  - 写回 (write-back): 尽可能迟地更新, 只有当替换算法将这个更新过的块要替换出去时才把它写到紧接着的低一层中。该策略利用了局部性原理, 能显著减少总线流量; 缺点是显著增加了复杂性。高速缓存必须为每一个高速缓存行维护一个额外的修改位 (dirty bit), 表明这个块是否被修改过。
- 写不命中:
  - 写分配 (write-allocate): 加载相应的低一层的块到 Cache 中并更新这个 Cache 块。该方法利用了写的空间局部性, 但缺点是每次写不命中都会导致一个块从低一层传送到 Cachhe。
  - 非写分配 (not-write-allocate): 避开 Cache, 直接将这个字写到低一层中。
- 直写 Cache 通常是非写分配的; 写回 Cache 通常是写分配的。

4 实验过程

本实验在配置好环境的基础上, 通过 NEMU 获得示例程序 (hello.c, 即矩阵乘法) 的内存地址访问序列 (trace), 分别测试矩阵大小为 $32, 64, 128$ 下获得对应 3 组 trace.log 文件, 再通过 Cache 模拟器程序实现 cache 工作原理 (包括命中检查机制、内存地址和 cache 块的映射、替换算法等), 获取在不同参数、不同策略下的 miss/hit 变化情况, 最后分析并解释这些数据。具体过程如下:

4.1 Cache 大小对命中率的影响

4.1.1 实验结果

选取矩阵规模为 128 时的 trace 数据分析, 固定映射方法为 4 路组相联、替换算法为"LRU", 分别测试分组大小为 $16B, 64B, 128B$ 在 Cache 大小分别为 $1, 2, 4, 8, 16, 32, 64, 128KB$ 时的命中率, 结果如下: 对应折线图如图:

Table 1: Cache 大小对命中率的影响

Cache 大小 (KB)	块大小 (B)	命中率 (%)	Miss 数
1	16	48.934697591460555	2171337
2	16	49.42544576429553	2150470
4	16	49.43146634857913	2150214
8	16	49.43974465196907	2149862
16	16	49.457806404819856	2149094
32	16	49.493929910521416	2147558
64	16	96.83107957307472	134745
128	16	99.42044820898201	24643
1	64	49.29104092374577	2156185
2	64	49.74862884720627	2136728
4	64	49.75163913934807	2136600
8	64	49.75239171238352	2136568
16	64	49.75765972363166	2136344
32	64	49.76650245679819	2135968
64	64	97.26987668855635	116087
128	64	99.85480043997302	6174
1	128	49.2675700522027	2157183
2	128	49.76130499927212	2136189
4	128	49.76431529141392	2136061
8	128	49.76431529141392	2136061
16	128	49.76431529141392	2136061
32	128	49.77033587569751	2135805
64	128	97.14652526446474	121332
128	128	99.92725911254236	3093

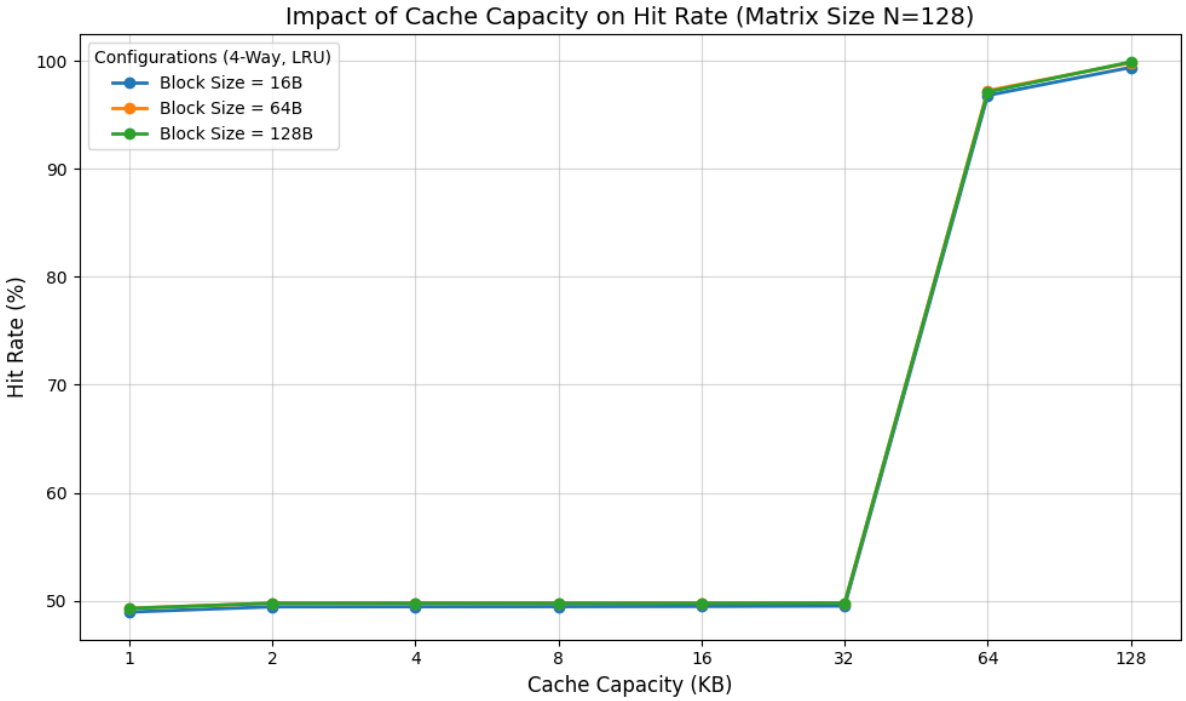

Figure 1: Cache 大小对命中率的影响

4.1.2 实验现象描述与分析

- 数据基础: 在 $N = 128$ 的 int 型矩阵乘法中
  - 单行大小:$128 \times 4Bytes = 512Bytes$
  - 单个矩阵大小:$128 \times 128 \times 4Bytes = 64KB$
  - 三个矩阵总大小 (A,B,C):$64KB \times 3 = 192KB$
- 阶段性现象与原因分析:
  - 低容量阶段 ($1KB \sim 32KB$)
    - 现象描述: 命中率极低且极其稳定, 基本维持在 49% 左右, 即使 Cache 容量从 1KB 增加到 32KB(增加了 32 倍), 命中率几乎没有实质性提升,Miss 数居高不下。
    - 原因分析:
      - 数据量远超容量: 此时 Cache 容量最大仅为 $32KB$, 而单个矩阵大小为 $64KB$。这意味着 Cache 连一个完整的矩阵都无法容纳。
      - 矩阵 B 的破坏性访问: 矩阵乘法中,B 矩阵是按列访问的 ($B[k][j]$), 其访存步长为 512Bytes。在容量小于 $64KB$ 时, 每当外层循环切换，之前缓存在 Cache 中的 B 矩阵行数据会因为容量不足而被后续数据替换, 产生大量的 Cache Miss。
      - 命中率的构成：此阶段的命中主要来源于矩阵 A 的行优先访问 (利用了空间局部性) 和矩阵 C 的重复累加 (利用了时间局部性)，这两者约占总访存的一半，故命中率固定在 49% 附近。

- 性能跃迁 ($32KB \to 64KB$)
  - 现象描述: 当容量从 $32KB$ 增加到 $64KB$ 时，命中率从 49.7% 瞬间飙升至 95% 以上, 从图像上看图线的斜率陡然增加,Cache 性能大幅上升。
  - 原因分析:
    - 数据容纳:$64KB$ 正好等于一个 int 型矩阵 ($128 \times 128 \times 4$) 的大小。当 Cache 容量达到 64KB 时, 系统恰好能够完整地缓存下矩阵 A 或矩阵 B 的绝大部分活跃数据。
    - 循环间访存失效消除: 此时, 在内层循环中被加载的 B 矩阵块可以驻留在 Cache 中供后续循环使用。这种从“无法容纳”到“完全容纳”的改变，消除了绝大部分的重复容量失效，导致了命中率的断崖式上升。
- 饱和稳定阶段 ($64KB \sim 128KB$)
  - 现象描述: 命中率进一步小幅度提升并最终稳定在 99.8% 左右。
  - 原因分析:
    - 数据完全容纳: 当容量达到 128KB 时，已非常接近总数据量 ($192KB$) 的量级。此时除了最初加载数据产生的不可避免的冷启动失效 (Cold Miss) 外, 几乎所有的 Cache Miss 都消失了。
    - 边际效益递减：由于核心数据集已被完全容纳, 继续增加容量对命中率的提升空间极小, 性能达到硬件优化上限。

- 块大小 (Block Size) 现象分析:
  - 现象描述: 在数据表中，相同的容量下，改变 Block Size($16B, 64B, 128B$) 对结果的影响非常微弱, 基本可以忽略不计。
  - 原因分析: 矩阵 B 的访存步长为 512B, 这远大于我们测试的最大块大小 (128B)。因此, 增大块大小无法通过预取机制帮矩阵 B 找回数据，无法利用其空间局部性。

4.1.3 实验总结

通过对 int 矩阵的测试可以得出, 对于特定的算法和矩阵规模,Cache 大小存在性能阈值, 该阈值与单个矩阵大小有密切关系; 非优化的矩阵乘法由于存在跨行访存 (B 矩阵), 其对 Cache 块大小并不敏感, 性能的提升完全依赖于 Cache 总容量是否能覆盖 B 的活跃数据。